NoSQL

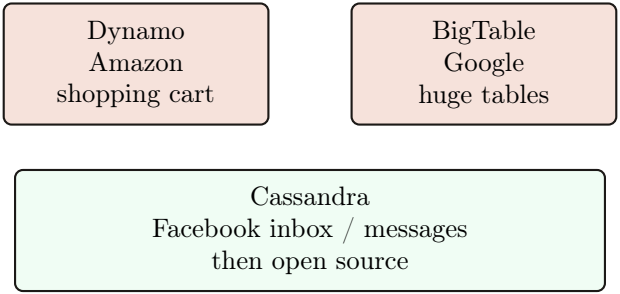
Analysis and Assessment of Design Quality

What is NoSQL?

Databases built for many machines
and flexible data shapes.

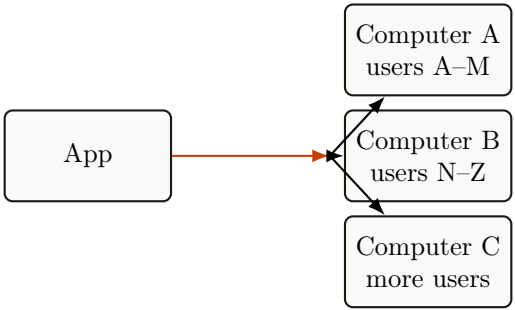
Not “better than SQL” — different job.

Three famous roots



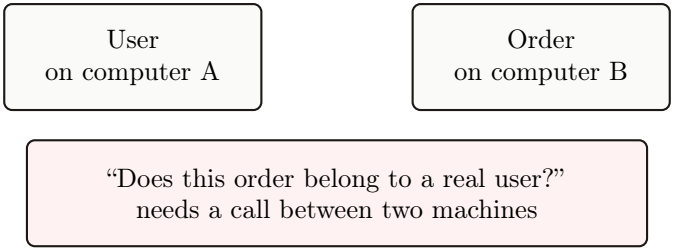
Same need: too much data for one SQL server.

Sharding = split data across computers



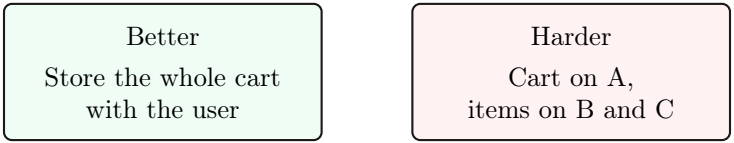
Shard = one slice of the data on one machine.

Foreign keys hate sharding



On one DB that check is easy. Across machines it is slow and fragile.

Shopping cart — keep things together



Put related data on the same machine when you can.

CAP in plain words

C — everyone sees the same latest value

A — the system always answers

P — it still runs when the network breaks

Story — two warehouses, phone line down



Both still get orders. They cannot check each other.

What do you do?

Stop selling	Keep selling
Wait until the line is back	Fix stock numbers later
safer stock	cart still works
some customers wait	may oversell

That is the CAP trade: same truth vs always open.

Shopping cart often picks “keep open”

Cart still accepts items
even if copies disagree for a while.

Later the systems catch up. Money / bank balance is a different story.

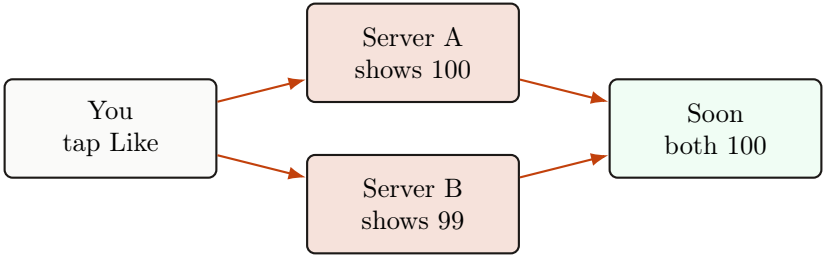
BASE — three ideas

Basically Available — keep answering users

Soft state — copies may differ for a short time

Eventually consistent — they agree later

Story — like count on a social feed



Fine for likes. Bad for “did I pay twice?”

ACID vs BASE — one line each

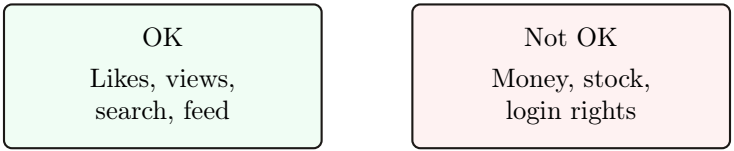
ACID

Correct right now
(deck 12)

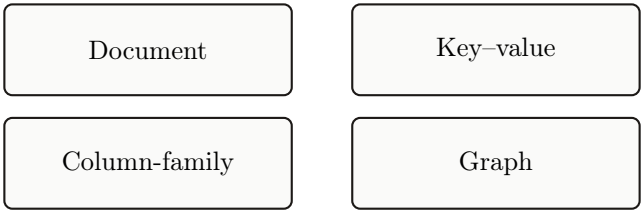
BASE

Stay up;
agree later

OK vs not OK for “agree later”



Four kinds of NoSQL



Next: one slide each — what it is + when to use it.

Document

Stores JSON-like objects (e.g. a product page)

Use when: flexible fields — product catalog, user profile

Example tools: MongoDB.

Key–value

Ask by id: session:42 → get the value

Use when: cache, shopping cart by user id, sessions

Example tools: Redis, DynamoDB.

Column-family

Wide rows built for big clusters and many writes

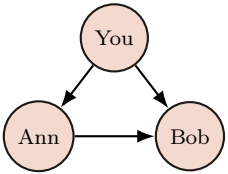
Use when: event logs, click streams, message history

Example tool: Cassandra.

Graph

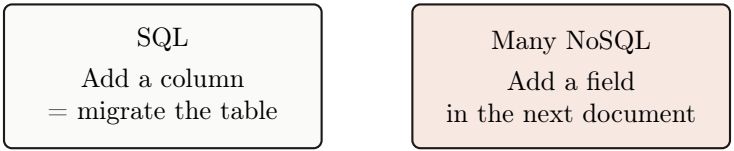
People and links between them

Use when: friends-of-friends, recommendations



Example tool: Neo4j.

Schema can change easily



Example: product gets a new “colour” field — no big table rewrite.

Does it scale by itself?

The DB helps with copies and splitting data

You still design keys and queries — not magic

Not better — different

Pick the tool for the job.

SQL and NoSQL can live in the same product.

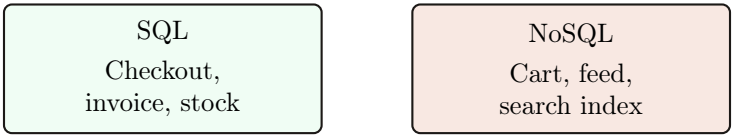
How to choose

Need money-safe rules? → lean SQL / ACID

Need flexible docs or huge write scale? → lean NoSQL

Need both? → use both (cart vs payment)

Quick examples



Quiz — cart vs payment

Quiz

The shopping cart must stay up if the network splits. Payment must never charge twice. What do you do?

1. One SQL database for everything
2. Soft / available store for the cart; strict ACID for payment
3. Graph database for both
4. Key–value store for payment only

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — cart vs payment

Answer

2. Cart can be a bit soft; payment must be exact — different rules, often different stores.

Remember

NoSQL = many machines + flexible data

Network break: same truth vs stay open

BASE = answer now, agree later

Four types: pick by access pattern

Architect takeaway

Use NoSQL where scale or flexible shape matters

Keep related data together — foreign keys fight sharding

Keep ACID for money and stock — do not fashion-pick